# STAT 432 Homework 05

Conner Jensen (connerj2)

## Contents

**Question 1: Bias and variance of KNN** **1**

(a) . . . 3

(b) . . . 3

**Question 2: KNN versus lasso** **3**

(a) . . . 4

(b) . . . 4

(c) . . . 5

**Question 3: Observed dimension and latent dimension** **5**

(a)–(b) . . . 6

(c) . . . 6

**Question 4: Classify handwritten digits** **7**

(a)–(b) . . . 9

(c) . . . 9

**Question 5: Plot-formatting skill** **9**

**`plot-formatting` skill** **10**

## Question 1: Bias and variance of KNN

The true mean at $\mathbf{x}_0 = (0.5, 0.7, 1)^\mathsf{T}$ is $f(\mathbf{x}_0) = 0.5 \cdot 0.5 + \sin(0.7) - 0.3 \cdot 1^2$. Seed `43251` is used for the single training draw in (a) and for the 200 independent draws in (b).

```
set.seed(43251)
x0 <- matrix(c(0.5, 0.7, 1), nrow = 1)
f0 <- f_mean(x0)
ks <- seq(1, 29, by = 2)
n <- 400
nsim <- 200
pred_mat <- matrix(NA_real_, nsim, length(ks))
```

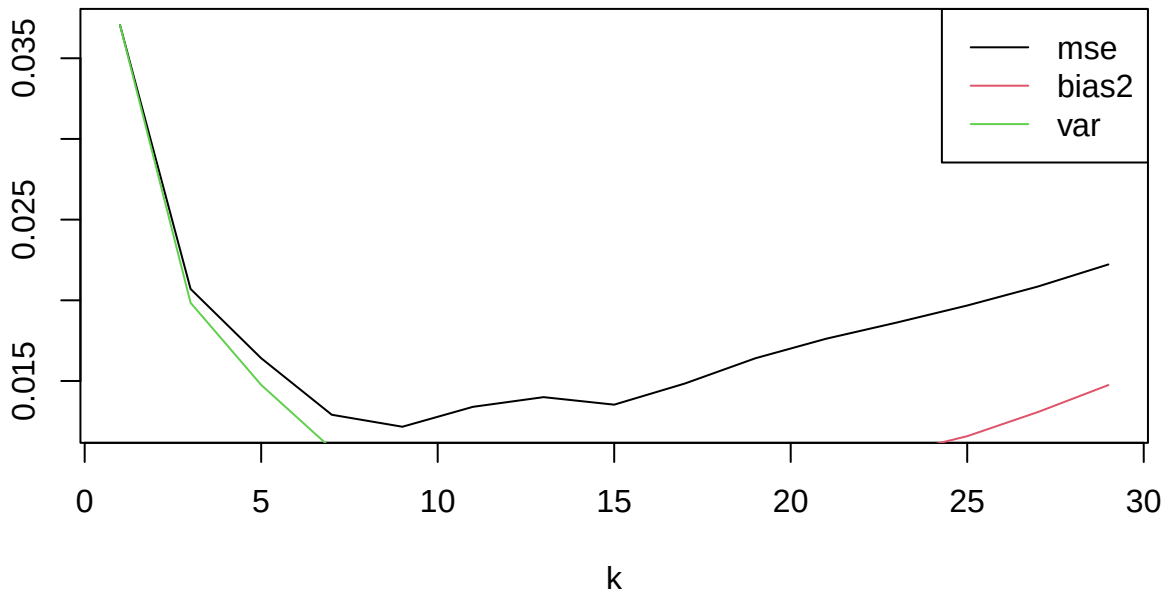
```r
for (s in seq_len(nsim)) {
  X <- matrix(rnorm(n * 3), n, 3)
  y <- f_mean(X) + rnorm(n, 0, 0.1)
  for (j in seq_along(ks)) {
    pred_mat[s, j] <- knn_reg(X, y, x0, ks[j])
  }
}
mhat <- colMeans(pred_mat)
bias <- mhat - f0
vari <- colMeans((pred_mat - matrix(mhat, nsim, length(ks), byrow = TRUE))^2)
mse <- colMeans((pred_mat - f0)^2)
c(f0 = f0, max_decomp_err = max(abs(mse - bias^2 - vari)))
```

```
##             f0 max_decomp_err
##   5.942177e-01   5.377643e-17
```

```r
data.frame(
  k = ks,
  pred_one = pred_mat[1, ],
  sq_bias = bias^2,
  variance = vari,
  mse = mse
)
```

```
##     k  pred_one      sq_bias    variance        mse
## 1   1 0.7888302 9.370024e-07 0.037061764 0.03706270
## 2   3 0.6621674 8.564462e-04 0.019848502 0.02070495
## 3   5 0.6642853 1.655021e-03 0.014757089 0.01641211
## 4   7 0.6100117 2.111558e-03 0.010794794 0.01290635
## 5   9 0.6318671 2.665047e-03 0.009499078 0.01216412
## 6  11 0.4925114 4.013807e-03 0.009385895 0.01339970
## 7  13 0.5222989 4.689804e-03 0.009309224 0.01399903
## 8  15 0.5810203 4.985708e-03 0.008547858 0.01353357
## 9  17 0.5738171 6.611781e-03 0.008229549 0.01484133
## 10 19 0.5751479 7.960509e-03 0.008447428 0.01640794
## 11 21 0.5800517 9.171688e-03 0.008444994 0.01761668
## 12 23 0.5519177 1.049846e-02 0.008120898 0.01861936
## 13 25 0.5214556 1.158040e-02 0.008096522 0.01967693
## 14 27 0.4952957 1.307386e-02 0.007779123 0.02085298
## 15 29 0.4972667 1.474784e-02 0.007474509 0.02222235
```

```r
plot(ks, mse, type = "l", xlab = "k", ylab = "")
lines(ks, bias^2, col = 2)
lines(ks, vari, col = 3)
legend("topright", c("mse", "bias2", "var"), col = 1:3, lty = 1)
```




### (a)

$f(\mathbf{x}_0) \approx 0.594$. The first training sample's 5NN prediction is about 0.66; $k = 1$ is farther away (0.79) and large $k$ pulls the prediction down toward 0.50.

### (b)

The identity MSE $=$ bias$^2$ $+$ Var holds up to roundoff (divisor 200 for the variance). Squared bias rises with $k$ because farther neighbors are averaged in. Variance falls quickly from $k = 1$ to about $k = 9$ and then levels off. MSE is U-shaped and smallest near $k = 9$.

This is the same bias–variance trade-off as ridge or lasso, but the knob runs the other way: large $k$ is more smoothing, like a large $\lambda$. Ridge and lasso shrink a global linear fit; KNN averages nearby responses and does not penalize coefficients.

## Question 2: KNN versus lasso

Seed `43252` is used for both settings, generated in order. Lasso uses the Week 4 objective through `glmnet` with an unpenalized intercept, lecture scaling (divisor equal to the training-fold sample size) recomputed inside each of 10 folds, and $\lambda \in \{10^0, 10^{-0.1}, \ldots, 10^{-4}\}$.

```
set.seed(43252)
Lambda <- 10^seq(0, -4, length.out = 41)
ntr <- 400
nte <- 1000
```

```r
p <- 30

run_setting <- function(corr) {
  if (!corr) {
    Xtr <- matrix(rnorm(ntr * p), ntr, p)
    Xte <- matrix(rnorm(nte * p), nte, p)
  } else {
    Sigma <- matrix(0.8, p, p)
    diag(Sigma) <- 1
    R <- chol(Sigma)
    Xtr <- matrix(rnorm(ntr * p), ntr, p) %*% R
    Xte <- matrix(rnorm(nte * p), nte, p) %*% R
  }
  ytr <- f_mean(Xtr) + rnorm(ntr, 0, 0.1)
  yte <- f_mean(Xte) + rnorm(nte, 0, 0.1)
  pred_knn <- knn_reg(Xtr, ytr, Xte, 5)
  folds <- sample(rep(1:10, length.out = ntr))
  cv <- lasso_cv(Xtr, ytr, Lambda, folds)
  las <- lasso_fit_pred(Xtr, ytr, Xte, cv$lambda)
  data.frame(
    knn_mse = mean((yte - pred_knn)^2),
    lasso_mse = mean((yte - las$pred)^2),
    lasso_lambda = cv$lambda,
    lasso_nzero = las$nzero
  )
}

q2_tab <- rbind(
  independent = run_setting(FALSE),
  correlated = run_setting(TRUE)
)
q2_tab
```

```
##                knn_mse lasso_mse lasso_lambda lasso_nzero
## independent 0.5564549 0.2454593   0.03981072           4
## correlated  0.1641018 0.2340070   0.01584893           9
```

## (a)

The table above reports test MSE for 5NN and lasso, the CV-selected $\lambda$, and the number of nonzero lasso slopes.

## (b)

With independent covariates, 5NN is much worse than lasso. Euclidean distance uses all 30 coordinates equally, so 27 noise variables swamp the three that enter $f$. Lasso does not treat coordinates that way: it keeps a sparse linear subset (here 4 slopes) and zeros the rest.

With correlation 0.8, every coordinate carries some information about the others, so neighborhoods in $\mathbf{x}$-space become more meaningful and 5NN improves a lot. Lasso still uses only a few proxies rather than the whole correlated block.

### (c)

The lasso model is linear in the 30 covariates, so it cannot represent $\sin(X_2)$ or $X_3^2$. Some of its error is misspecification, not just penalty choice. KNN can follow those curves locally, but only if the neighborhoods are actually close in the relevant coordinates.

These two draws go in opposite directions (lasso wins when the covariates are independent; 5NN wins when they are correlated), so they do not support a claim that correlation always helps KNN or that one method is always better.

## Question 3: Observed dimension and latent dimension

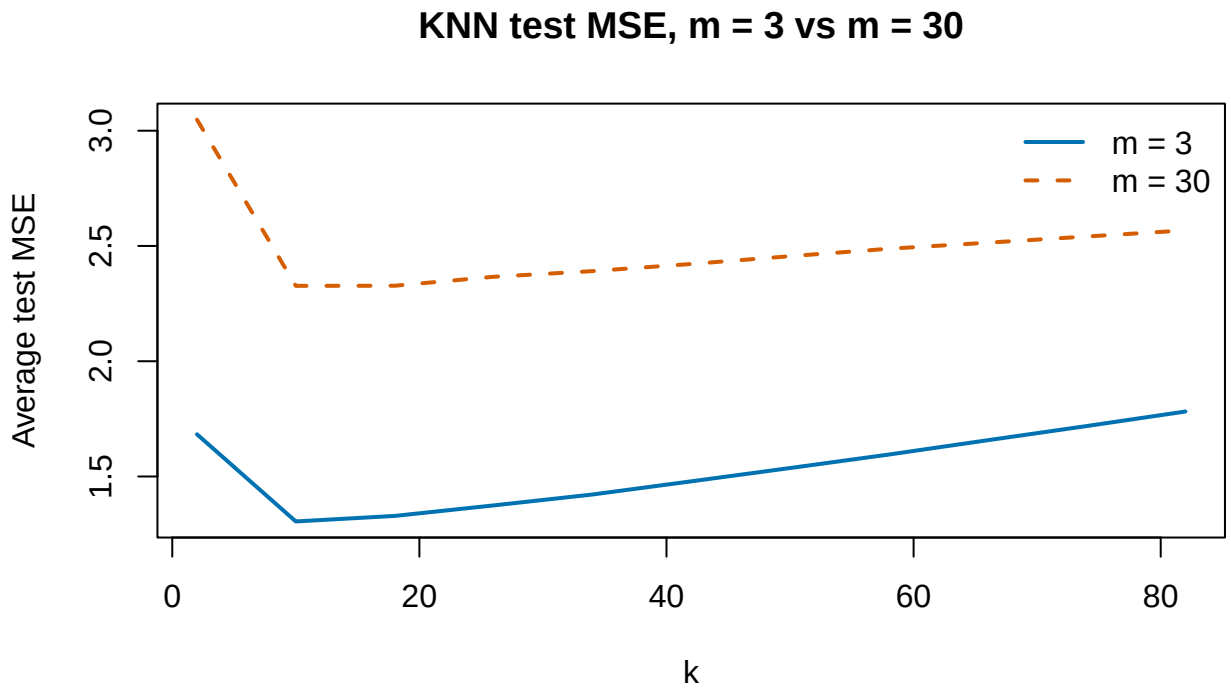
Seed 43254. Each repetition draws a new $\mathbf{A}$ and new training and test samples. KNN sees only the 100 observed columns of $\mathbf{X}$.

```
set.seed(43254)
ms <- c(3, 30)
ks3 <- seq(2, 82, by = 8)
nrep <- 50
ntr <- 200
nte <- 200
p <- 100
mse3 <- array(NA_real_, c(nrep, length(ms), length(ks3)))
for (r in seq_len(nrep)) {
  for (im in seq_along(ms)) {
    m <- ms[im]
    A <- matrix(runif(p * m, -2, 2), p, m)
    Ztr <- matrix(rnorm(ntr * m), ntr, m)
    Zte <- matrix(rnorm(nte * m), nte, m)
    Xtr <- Ztr %*% t(A) + matrix(rnorm(ntr * p), ntr, p)
    Xte <- Zte %*% t(A) + matrix(rnorm(nte * p), nte, p)
    ytr <- Ztr[, 1] + Ztr[, 2] + rnorm(ntr)
    yte <- Zte[, 1] + Zte[, 2] + rnorm(nte)
    d2 <- outer(rowSums(Xte^2), rowSums(Xtr^2), "+") -
      2 * Xte %*% t(Xtr)
    ord <- t(apply(d2, 1, function(v) order(v, seq_len(ntr))))
    for (j in seq_along(ks3)) {
      nn <- ord[, seq_len(ks3[j]), drop = FALSE]
      pred <- rowMeans(matrix(ytr[nn], nrow = nte))
      mse3[r, im, j] <- mean((yte - pred)^2)
    }
  }
}
avg_mse <- apply(mse3, c(2, 3), mean)
data.frame(k = ks3, m3 = avg_mse[1, ], m30 = avg_mse[2, ])
```

```
##    k       m3      m30
## 1  2 1.683127 3.047850
## 2 10 1.304908 2.327015
## 3 18 1.328875 2.327491
## 4 26 1.374519 2.366365
## 5 34 1.421772 2.390902
```

```
## 6  42 1.478920 2.421906
## 7  50 1.536628 2.455616
## 8  58 1.595212 2.487913
## 9  66 1.657265 2.514130
## 10 74 1.718717 2.541038
## 11 82 1.781810 2.567607
```

```r
plot(ks3, avg_mse[1, ], type = "l", lwd = 2, col = "#0072B2",
     ylim = range(avg_mse),
     xlab = "k", ylab = "Average test MSE",
     main = "KNN test MSE, m = 3 vs m = 30")
lines(ks3, avg_mse[2, ], lwd = 2, lty = 2, col = "#D55E00")
legend("topright", bty = "n",
       legend = c("m = 3", "m = 30"),
       col = c("#0072B2", "#D55E00"), lwd = 2, lty = c(1, 2))
```




## (a)–(b)

For every listed $k$, average test MSE is smaller when $m = 3$ than when $m = 30$. Both curves improve from $k = 2$ to $k = 10$ and then rise as $k$ grows.

## (c)

Both settings have 100 observed covariates, and $Y$ depends on only two latent variables. When $m = 3$, those two directions are a large part of the latent space, so neighbors in $\mathbf{X}$ tend to be neighbors in $(Z_1, Z_2)$. When $m = 30$, distances are dominated by 28 unused latents, and KNN matches on the wrong coordinates.

The noise $\boldsymbol{\eta}_i$ keeps $\mathbf{X}_i$ from lying in the column space of $\mathbf{A}$. Without it, the observed points would sit in an exact $m$-dimensional linear subspace. With it, they form a 100-dimensional cloud around that subspace, so Euclidean neighborhoods are noisier in both settings.

## Question 4: Classify handwritten digits

The first 1,000 training rows and first 500 test rows of `data/knn/zip-train.csv.gz` and `zip-test.csv.gz` are used, with original pixel values and no label in the distance.

```
knn5_class <- function(Xtr, ytr, Xte, manhattan = FALSE) {
  ntr <- nrow(Xtr)
  nte <- nrow(Xte)
  pred <- integer(nte)
  idx <- seq_len(ntr)
  for (i in seq_len(nte)) {
    d <- if (manhattan) {
      rowSums(abs(sweep(Xtr, 2, Xte[i, ], "-")))
    } else {
      sqrt(rowSums(sweep(Xtr, 2, Xte[i, ], "-")^2))
    }
    nn <- order(d, idx)[1:5]
    counts <- tabulate(ytr[nn] + 1L, nbins = 10L)
    pred[i] <- which.max(counts) - 1L
  }
  pred
}
```

Distance ties use the smaller training row number; voting ties use the smallest digit (`which.max` on counts for digits $0, \ldots, 9$).

```
tr <- as.matrix(read.csv("data/knn/zip-train.csv.gz", header = FALSE))
te <- as.matrix(read.csv("data/knn/zip-test.csv.gz", header = FALSE))
tr <- tr[1:1000, ]
te <- te[1:500, ]
ytr <- as.integer(tr[, 1])
yte <- as.integer(te[, 1])
Xtr <- tr[, -1]
Xte <- te[, -1]
pred_e <- knn5_class(Xtr, ytr, Xte, FALSE)
pred_m <- knn5_class(Xtr, ytr, Xte, TRUE)
c(euclid_error = mean(pred_e != yte),
  manhattan_error = mean(pred_m != yte),
  pred_disagree = mean(pred_e != pred_m))
```

```
##    euclid_error manhattan_error   pred_disagree
##           0.122           0.138           0.044
```

```
cm_e <- table(
  pred = factor(pred_e, 0:9),
  true = factor(yte, 0:9)
)
```

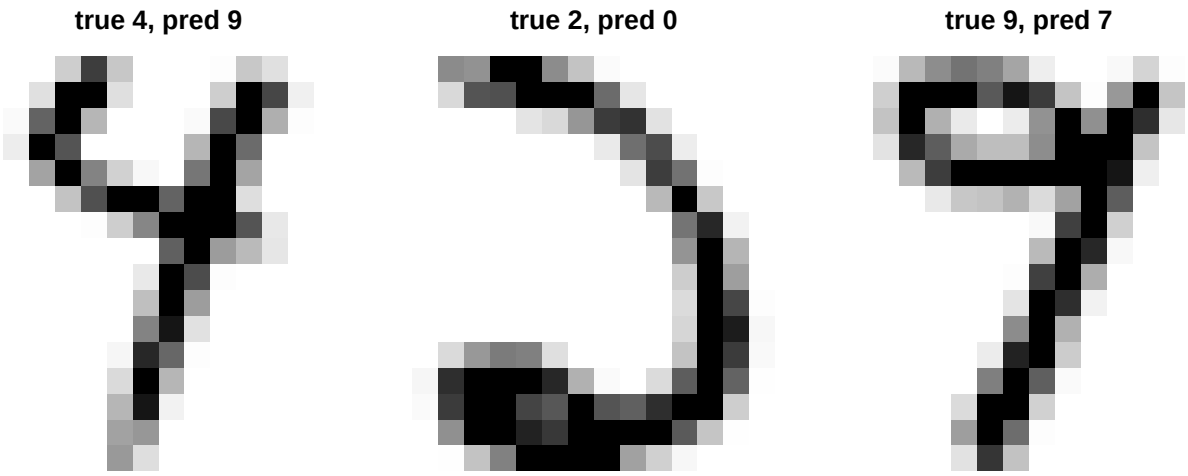
```r
cm_m <- table(
  pred = factor(pred_m, 0:9),
  true = factor(yte, 0:9)
)
cm_e
```

```
##     true
## pred   0   1   2   3   4   5   6   7   8   9
##    0 118   0   7   1   0   4   2   0   2   0
##    1   0  66   0   0   2   0   0   0   0   1
##    2   0   0  39   1   0   0   2   0   0   1
##    3   0   0   0  23   0   4   0   0   2   0
##    4   0   0   1   0  27   1   1   1   0   1
##    5   0   0   0   1   0  13   0   0   0   0
##    6   0   2   0   0   0   1  41   0   2   0
##    7   0   0   0   0   1   1   0  34   0   4
##    8   0   0   3   2   0   2   0   0  37   0
##    9   1   0   0   1   4   0   0   1   1  41
```

```r
cm_m
```

```
##     true
## pred   0   1   2   3   4   5   6   7   8   9
##    0 118   0   6   1   0   5   3   0   4   0
##    1   0  67   2   0   2   0   1   0   0   1
##    2   0   0  35   0   0   0   2   0   0   0
##    3   0   0   1  24   0   4   0   0   2   0
##    4   0   0   1   1  27   1   0   1   0   1
##    5   0   0   0   1   0  11   0   0   0   0
##    6   0   1   0   0   0   2  40   0   2   0
##    7   0   0   2   1   1   0   0  34   0   5
##    8   0   0   2   0   0   3   0   0  34   0
##    9   1   0   1   1   4   0   0   1   2  41
```

```r
id <- c(
  which(yte == 4 & pred_e == 9)[1],
  which(yte == 2 & pred_e == 0)[1],
  which(yte == 9 & pred_e == 7)[1]
)
par(mfrow = c(1, 3), mar = c(0.4, 0.4, 2.2, 0.4))
for (i in id) {
  show_digit(
    Xte[i, ],
    main = sprintf("true %d, pred %d", yte[i], pred_e[i])
  )
}
```




### (a)–(b)

Euclidean 5NN has test error 0.122 (61/500). Manhattan has test error 0.138 (69/500). The largest off-diagonal counts are 2 predicted as 0, 4 predicted as 9, 9 predicted as 7, and 5 predicted as 0 or 3. Those pairs are the most commonly confused digits.

### (c)

The three Euclidean mistakes above are visually understandable: a closed 4 can look like a 9, an open 2 with no bottom loop can look like a 0, and a 9 with a straight tail can look like a 7.

The two distances have similar total error but are not the same classifier. They disagree on 0.044 of the 500 images (22 cases), including images where both are wrong but the predicted digits differ. Similar error rates therefore do not mean the same mistakes.

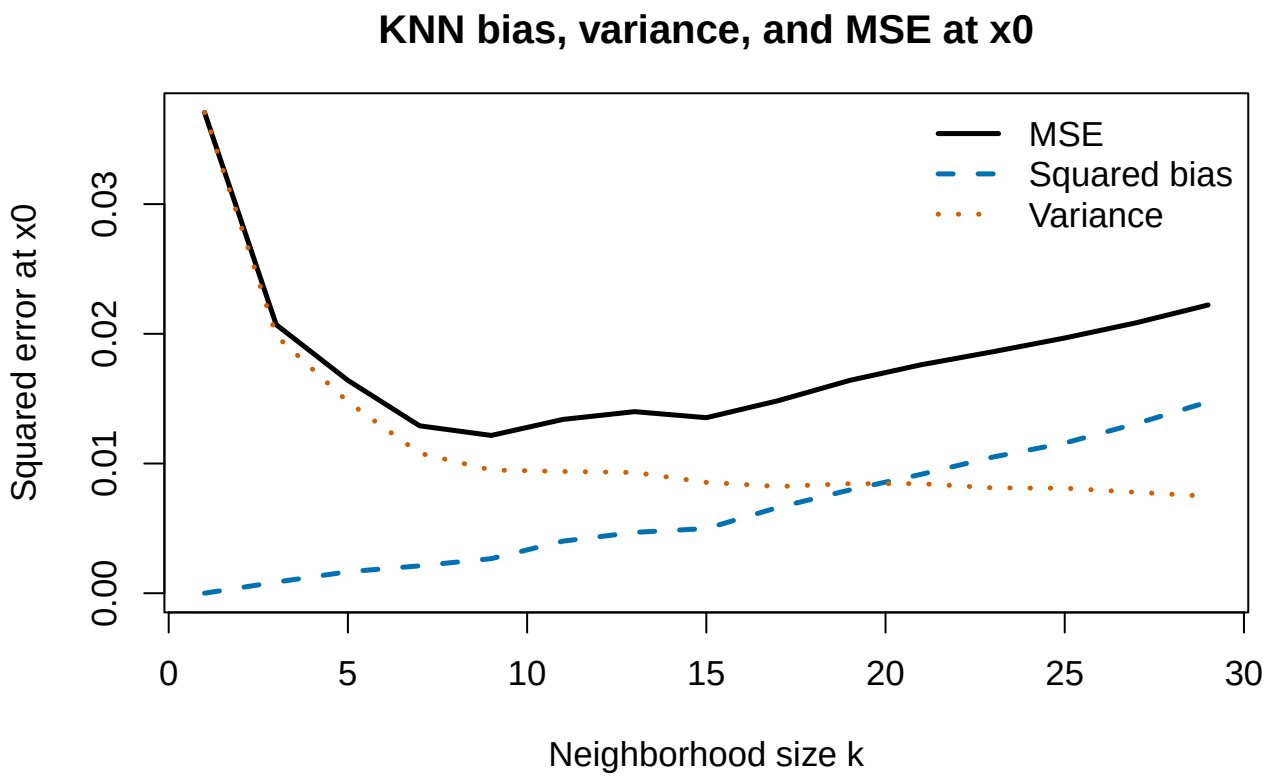
## Question 5: Plot-formatting skill

I wrote `plot-formatting` and installed it at `.cursor/skills/plot-formatting/SKILL.md`. It only runs when asked explicitly, and it is limited to STAT 432 homework plot formatting.

I then asked the agent to use it on the Question 1 figure. The original plot (Question 1) uses default margins, one line type, and the default palette. The revision below keeps the same three series and follows the skill: more margin, a comparison title, axis labels, a legend, color plus line type, and thicker lines.

```
par(mar = c(4.4, 4.4, 3.2, 1.1), cex = 1.05)
cols <- c("#000000", "#0072B2", "#D55E00")
ltys <- c(1, 2, 3)
yl <- range(c(mse, bias^2, vari))
plot(ks, mse, type = "l", lwd = 2.5, col = cols[1], lty = ltys[1],
     ylim = yl, xlab = "Neighborhood size k",
     ylab = "Squared error at x0",
     main = "KNN bias, variance, and MSE at x0")
lines(ks, bias^2, lwd = 2.5, col = cols[2], lty = ltys[2])
lines(ks, vari, lwd = 2.5, col = cols[3], lty = ltys[3])
legend("topright", bty = "n",
```

```
       legend = c("MSE", "Squared bias", "Variance"),
       col = cols, lty = ltys, lwd = 2.5)
```




The revised figure is easier to read in the PDF: the three curves stay distinguishable in grayscale, and the axis labels say what is being plotted. I kept this version of the skill.

## `plot-formatting` skill

```
---
name: plot-formatting
description: >
  Improve formatting of STAT 432 homework plots in R or Python.
  Use only when the student explicitly asks to use plot-formatting
  on a homework figure.
disable-model-invocation: true
---

# Format a STAT 432 homework plot

Revise an existing homework figure so it is readable in a PDF
report. Do not change the statistical content: same data, same
estimator, same comparison.
```

```
When asked to use this skill on a plot:

1. **Margins.** Leave enough margin for the title, axis titles,
   tick labels, and a legend. Do not crop text.
2. **Title.** Use a short title that states the comparison, not
   a generic label such as "Plot 1".
3. **Axes.** Label both axes with the plotted quantity and any
   symbol or unit. Keep tick labels large enough to read.
4. **Labels.** Identify every series in a legend or direct
   labels. Do not rely on color alone: also vary line type or
   plotting character.
5. **Colors.** Use a small colorblind-friendly set with strong
   contrast on a white background.
6. **Sizes.** Use PDF-friendly dimensions (about 6.5--7 inches
   wide), line width at least 2, and point/text size that
   remains legible after shrinking.

Keep the rules portable across R (`plot`, `ggplot2`) and Python
(`matplotlib`). Stop after revising the figure. Do not add
analysis that the homework did not ask for.
```